

CSED312 Project 1

Design report

팀 오우노

20230355 정지성

20230345 이성재

목차

1. 현재의 pintos implementation에 대한 분석

1.1. Loading 과정

1.2. Threads의 구현에 대한 이해

1.3. Synchronization Primitives

2. Design Proposal

2.1. Alarm clock

2.2. Priority Scheduling

2.3. Advanced Scheduler

1. 현재의 pintos implementation에 대한 분석

1.1. Loading 과정 (최초 실행부터 idle thread 생성까지)

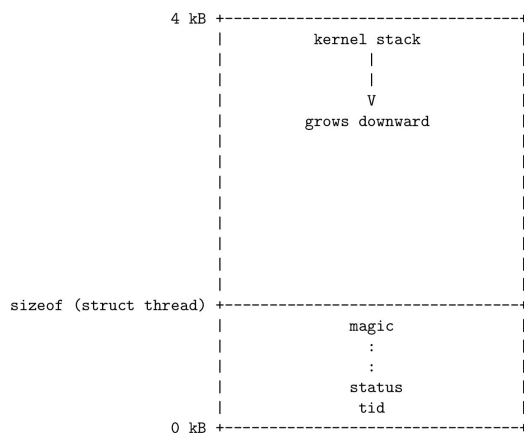
Pintos를 맨 처음 실행시켰을 때, PC BIOS가 로더 프로그램 `threads/loader.S`을 메모리에 load한다.

`loader.S`의 역할은 디스크에서 커널을 찾고 이를 메모리에 load한 뒤, 커널의 entry point 주소로 jump하는 역할을 한다. 이렇게 해서 실행되는 함수가 `threads/start.S`의 `start()`이며, 이 파일은 기기의 메모리 크기 확인, page table initialization 등의 low-level kernel initialization을 진행한다.

이를 완료하면 `threads/init.c` 파일의 `main()` 함수로 점프하여, 이 파일에서 high-level kernel initialization이 시작된다. `bss_init()`, `thread_init()`, `console_init()`, ... 등등 이후 커널의 실행에 필요한 정말 많은 것에 대한 초기화가 진행되고, 이 과정에서 idle thread가 생성된다. 마지막으로 `run_actions()`에서 실행 시 받은 command line input에 따라서 action들을 실행한다.

1.2. Threads의 구현에 대한 기본적인 이해

Pintos에서 각 thread는 4KiB의 고정된 메모리 페이지를 가지며, 주소가 가장 작은 쪽의 공간은 `struct thread`가 차지하고 해당 thread에 대한 여러 가지 중요한 정보를 저장한다. 또, 나머지 공간은 그 thread의 스택으로 활용되며 높은 주소에서 낮은 주소 방향으로 커진다.



앞으로 thread에 대해 다음과 같은 내용을 살펴 볼 것이다.

- 1.2.1. `struct thread`에 저장되는 정보
- 1.2.2. Thread와 관련되어 정의된 함수
- 1.2.3. Context switching 메커니즘
- 1.2.4. Context switching: new thread인 경우

1.2.1. struct thread에 저장되는 정보

우선 `threads/thread.h`에 정의된 `struct thread`를 살펴 보자.

```
struct thread
{
    tid_t tid;                /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16];            /* Name (for debugging purposes). */
    uint8_t *stack;           /* Saved stack pointer. */
    int priority;             /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */
    struct list_elem elem;    /* List element. */
#ifdef USERPROG
    uint32_t *pagedir;        /* Page directory. */
#endif
    unsigned magic;           /* Detects stack overflow. */
};
```

위에 정의된 각 member를 high level에서 분류해 보면, (a) thread의 식별을 위한 정보 (`tid`, `name`), (b) thread의 state에 대한 정보 (`status`), (c) scheduling 및 context switching에 필요한 정보 (`stack`, `priority`), (d) 그 외에 구현에 도움이 되는 것들 (`allelem`, `elem`, `pagedir`, `magic`)이 있다.

- `tid`: 각 thread는 고유한 tid를 가지고 있어야 한다.
- `status`: 아래의 네 가지 상태가 있다.
 - `THREAD_RUNNING`: thread가 실행중이다. 한 번에 하나의 thread만 이 상태일 수 있다.
 - `THREAD_READY`: 실행될 준비는 되어 있지만 실행은 되지 않고 있는 상태이다. 이 상태에 있는 모든 thread들은 `ready_list`에서 확인할 수 있다.
 - `THREAD_BLOCKED`: lock 등 무언가를 기다리느라 실행할 준비가 아직 되어 있지 않은 상태이다. 기다림이 끝나면 `THREAD_READY` 상태로 바뀐다.
 - `THREAD_DYING`: 곧 삭제될 thread이다.
- `stack`: 실행되고 있던 thread가 다른 thread로 전환되기 전에 `%esp`를 저장하는 곳이다. 자세한 내용은 1.2.3.에서 다루도록 하겠다.
- `priority`: thread의 중요도를 정수로 나타낸 정보이다. 현재 상태의 pintos 코드에서는 이 값을 무시하고 있으며, 이것을 이용해 스케줄러를 다시 짜는 것이 project 1의 두 번째 과제 내용이다.
- `magic`: 이를 이용하면 stack overflow가 발생했는지 감지할 수 있다.

1.2.2. Thread과 관련되어 정의된 함수

- thread_init()

```
void thread_init (void)
{
    ASSERT (intr_get_level () == INTR_OFF);

    lock_init (&tid_lock);
    list_init (&ready_list);
    list_init (&all_list);

    /* Set up a thread structure for the running thread. */
    initial_thread = running_thread ();
    init_thread (initial_thread, "main", PRI_DEFAULT);
    initial_thread->status = THREAD_RUNNING;
    initial_thread->tid = allocate_tid ();
}
```

`threads/init.c`의 `main()`에서 부르는 함수로, thread 시스템을 초기화한다. 역할은 (아직 high-level initialization중에 있는) kernel을 돌리고 있는 현재의 Pintos의 initial thread에 `struct thread`를 부여해, main thread로 만들어 주는 것이다. main thread가 initialize되어야 `thread_current()` 함수가 커널모드에서도 정상적으로 작동하는데, `thread_current()` 함수에 의존하는 다른 함수가 은근히 많다. 그렇기 때문에 `thread_init()`은 `main()`의 꽤 초반에서 실행된다.

- thread_start()

```
void
thread_start (void)
{
    /* Create the idle thread. */
    struct semaphore idle_started;
    sema_init (&idle_started, 0);
    thread_create ("idle", PRI_MIN, idle, &idle_started);

    /* Start preemptive thread scheduling. */
    intr_enable ();

    /* Wait for the idle thread to initialize idle_thread. */
    sema_down (&idle_started);
}
```

이 함수 또한 `threads/init.c`의 `main()`에서 부르며, 이 함수의 역할은 idle thread를 만드는 것이다. Interrupt가 처음으로 enable되는 곳이다. 이는 timer interrupt에 의한 스케줄러가 enable되기 시작하는 곳이라는 뜻이기도 하다. Semaphore을 이용해서, idle thread가 다 만들어질 때까지 함수를 벗어나지 않는 것을 확인할 수 있다.

- thread_create()

```
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    struct thread *t;
    struct kernel_thread_frame *kf;
    struct switch_entry_frame *ef;
    struct switch_threads_frame *sf;
    tid_t tid;

    ASSERT (function != NULL);

    /* Allocate thread. */
    t = palloc_get_page (PAL_ZERO);
    if (t == NULL)
        return TID_ERROR;
    /* Initialize thread. */
    init_thread (t, name, priority);
    tid = t->tid = allocate_tid ();

    /* Stack frame for kernel_thread(). */
    kf = alloc_frame (t, sizeof *kf);
    kf->eip = NULL;
    kf->function = function;
    kf->aux = aux;
    /* Stack frame for switch_entry(). */
    ef = alloc_frame (t, sizeof *ef);
    ef->eip = (void (*) (void)) kernel_thread;
    /* Stack frame for switch_threads(). */
    sf = alloc_frame (t, sizeof *sf);
    sf->eip = switch_entry;
    sf->ebp = 0;

    /* Add to run queue. */
    thread_unblock (t);
    return tid;
}
```

새 thread를 만들 때 사용되는 함수이다. `struct thread`에 `name`과 `priority`를 argument로 받아서 저장한다. `tid`도 설정해 준다. `init_thread()` 함수를 뜯어 보면 thread의 모든 바이트를 0으로 초기화하고, 초기 상태를 `THREAD_BLOCKED`로 설정하며 `stack`값과 `magic` 값도 초기화한다. 이후, `thread_create()` 함수에서 받은 argument들 중 `aux`을 argument로 하여 `function`을 실행하는 thread를 만든다. 마지막으로, `thread_unblock()` 함수에서 state를 `THREAD_READY`로 바꾸고 ready list에 넣어 준다. (이렇게 block에서 시작해서 unblock을 나중에 해 주는 이유는, creation이 전부 끝날 때까지 ready list에 넣지 않게 하기 위함이다.)

그런데 여기에 `kf`, `ef`과 `sf` 등 thread가 가져야 할 초기정보와는 관련없어 보이는 코드들이 조금 보인다. 이는 context switching과 밀접한 연관이 있는데, 더 자세한 내용은 1.2.4.에서 다루도록 하겠다.

- thread_block()

```
void
thread_block (void)
{
    ASSERT (!intr_context ());
    ASSERT (intr_get_level () == INTR_OFF);

    thread_current ()->status = THREAD_BLOCKED;
    schedule ();
}
```

여러 가지 sanity check을 한 후, 현재 실행 중인 thread의 state를 **THREAD_BLOCKED**로 만든다. 이후 **schedule()**을 불러 다음에 실행될 thread를 결정한다.

- thread_unblock()

```
void
thread_unblock (struct thread *t)
{
    enum intr_level old_level;

    ASSERT (is_thread (t));

    old_level = intr_disable ();
    ASSERT (t->status == THREAD_BLOCKED);
    list_push_back (&ready_list, &t->elem);
    t->status = THREAD_READY;
    intr_set_level (old_level);
}
```

THREAD_BLOCKED 상태에 있던 thread를 함수 argument로 받아, state를 **THREAD_READY**로 바꾸며 ready list에 추가해 준다. 이렇게 되면 thread가 다시 스케줄러에 의해 선택받을 수 있는 상태가 된다.

- thread_exit()

```
void
thread_exit (void)
{
    ASSERT (!intr_context ());

#ifdef USERPROG
    process_exit ();
#endif

    /* Remove thread from all threads list, set our status to dying,
       and schedule another process. That process will destroy us
       when it calls thread_schedule_tail(). */
    intr_disable ();
    list_remove (&thread_current()->allelem);
    thread_current ()->status = THREAD_DYING;
    schedule ();
    NOT_REACHED ();
}
```

현재 thread의 상태를 **THREAD_DYING**으로 설정하고, 전체 리스트에서도 thread를 삭제한 뒤 **schedule()** 함수를 부른다. Kernel이 이 thread를 다시 선택할 일이 없기 때문에 return하지 않는 함수이다.

- thread_yield()

```
void
thread_yield (void)
{
    struct thread *cur = thread_current ();
    enum intr_level old_level;

    ASSERT (!intr_context ());

    old_level = intr_disable ();
    if (cur != idle_thread)
        list_push_back (&ready_list, &cur->elem);
    cur->status = THREAD_READY;
    schedule ();
    intr_set_level (old_level);
}
```

주도권을 kernel의 스케줄러로 넘기는 함수라고 보면 된다. 현재 thread의 상태를 **THREAD_READY**로 설정하고 ready list에도 넣은 뒤 (idle thread인 경우 넣지 않음), **schedule()** 함수를 부른다. 이 때, **scheduler()**이 현재 thread를 바로 다시 부를 수 있으므로, 다른 thread를 돌리고 싶을 때에는 **thread_yield()**에 의존할 수 없다.

- thread_tick()

```
thread_tick (void)
{
    struct thread *t = thread_current ();

    /* Update statistics. */
    if (t == idle_thread)
        idle_ticks++;
#ifdef USERPROG
    else if (t->pagedir != NULL)
        user_ticks++;
#elseif
    else
        kernel_ticks++;

    /* Enforce preemption. */
    if (++thread_ticks >= TIME_SLICE)
        intr_yield_on_return ();
}
```

하나의 스레드가 너무 오래 cpu를 독점하지 못하도록 하는 라운드로빈 기법이 구현된 함수이다. 이 함수는 timer interrupt가 일어날 때마다 실행되는 함수이다. 함수의 첫 번째 조건문 부분은 스레드가 idle, user, kernel 상태에 각각 몇 tick 동안 있었는지 확인하기 위한 부분이다. 마지막 조건문이 핵심으로, `thread_ticks`을 1씩 증가시키며 그 값이 `TIME_SLICE` 보다 크거나 같아졌는지 확인한다. `TIME_SLICE`는 4로 정의 되어 있으며, 따라서 스레드가 4 tick동안 실행된 경우 강제로 스케줄러에게 양보하도록 `intr_yield_on_return()`을 호출한다. 즉, timer interrupt가 시간을 재는 역할을 하고 `thread_tick()` 함수가 시간이 오래 지난 경우 양보하도록 강제하는 역할을 한다고 볼 수 있다.

1.2.3. Context switching 메커니즘

기본적으로, 모든 context switching은 `schedule()` 함수 안에서 이루어진다. 이 함수는 `thread_block()`, `thread_exit()`과 `thread_yield()`의 세 함수 안에서만 호출되기 때문에, 그 이외의 코드에서 context switching이 일어날 걱정은 하지 않아도 된다. 각각의 함수가 `schedule()`을 호출하기 전에는 interrupt를 막아 둔 상태이며, 현재 돌리고 있던 thread의 상태를 `THREAD_RUNNING`이 아닌 다른 것으로 바꾸었다는 점을 염두에 두어야 한다. (이렇게 되면 `schedule()`는 현재 돌리던 thread의 state는 신경쓰지 않고 다음에 돌릴 thread의 state만 `THREAD_RUNNING`으로 바꾸면 된다.)

우선 `schedule()` 함수의 내부를 살펴 보자. 단순히 생각하기 위해, 현재 thread와 다음 thread 모두 새로운 thread가 아니라고 가정해 보자. (즉, `thread_create()`으로 생성된 이후 한 번이라도

`THREAD_RUNNING`상태였던 적이 있는 thread들)

```
static void
schedule (void)
{
    struct thread *cur = running_thread ();
    struct thread *next = next_thread_to_run ();
    struct thread *prev = NULL;

    ASSERT (intr_get_level () == INTR_OFF);
    ASSERT (cur->status != THREAD_RUNNING);
    ASSERT (is_thread (next));

    if (cur != next)
        prev = switch_threads (cur, next);
    thread_schedule_tail (prev);
}
```

`schedule()`은 현재 thread와 다음 thread의 포인터를 준비하고, 만약 현재와 다음 thread가 다르다면 context switching을 일으킨다. 먼저 `next_thread_to_run()` 함수를 보면서 다음 thread가 어떻게 선택되는지 관찰해 보자.

```
static struct thread *
next_thread_to_run (void)
{
    if (list_empty (&ready_list))
        return idle_thread;
    else
        return list_entry (list_pop_front (&ready_list), struct thread, elem);
}
```

위 함수를 보면, ready list가 비어 있는 경우 `idle_thread`를 schedule하고, 비어 있지 않은 경우 ready list의 제일 처음 entry를 schedule한다는 것을 볼 수 있다. 그런데 생각해 보면 우리는 ready list를 priority 순서대로 정렬해준 적이 없으므로, 이번 프로젝트에서 priority scheduling을 구현하기 위해서는 어떤 형태로든 정렬을 해야 할 것이다. (ex: `thread_unblock()`, `thread_yield()` 등에서 `list_push_back()`을 사용했는데, 이는 단순히 entry를 리스트의 맨 뒤에 추가하는 방식임)

다음으로, 실제 context switching의 중추적인 역할을 담당하는 `switch_threads()` 함수를 살펴 보자. 이 함수는 `threads/switch.S`에 정의되어 있다.

```
switch_threads:
    pushl %ebx
    pushl %ebp
    pushl %esi
    pushl %edi

    # Get offsetof (struct thread, stack).
.globl thread_stack_ofs
    mov thread_stack_ofs, %edx

    # Save current stack pointer to old thread's stack, if any.
    movl SWITCH_CUR(%esp), %eax
    movl %esp, (%eax,%edx,1)

    #####

    # Restore stack pointer from new thread's stack.
    movl SWITCH_NEXT(%esp), %ecx
    movl (%ecx,%edx,1), %esp

    # Restore caller's register state.
    popl %edi
    popl %esi
    popl %ebp
    popl %ebx
    ret
.endfunc
```

요약하면, 위 함수는 `#####`을 기준으로 현재와 다음 thread의 stack과 register이 교환되는 코드라고 할 수 있다. 먼저, 현재 thread가 자신의 stack에 `%ebx`, `%ebp` 등의 register을 저장한다. 그 후, `current_thread->stack`에 `%esp` 값을 저장한다. (이 위치에 `#####`이 온다.)

그 후, next_thread->stack 값을 불러와 %esp에 저장한다. 이후 stack에서 register값들을 다시 불러온다. 이렇게 되면 실질적으로 current_thread와 next_thread 간의 stack과 register이 교환되는 기능의 코드임을 알 수 있다.

그렇다면 %eip는 어떻게 새로운 thread에 적응할까? next_thread가 위 함수의 ret에 닿으면, stack에 적혀 있는 주소로 돌아가야 한다. 그런데 이 stack에 적혀 있는 주소는 next_thread가 예전에 switch_threads 함수로 점프를 하면서 적어 놓았을 주소이므로, %eip의 흐름 또한 current에서 next로 잘 넘어갔을 것임을 확인할 수 있다. 간단히 말해서, 결국 stack이 교환됐으므로 %eip의 흐름도 교환되었다.

한편, 지금까지의 논의는 next_thread가 예전에 switch_threads()를 통해 저장된 전적이 있었을 것임을 가정하고 진행되었다. 하지만 어떤 thread가 thread_create()로 생성된 이후 한 번도 running상태가 된 적이 없는 경우, (추가적 작업을 해 주지 않았다면) stack이 깨끗하기 때문에 stack에 register이 저장되어 있을 것이라는 가정도 할 수 없고, ret 이후의 return address도 적혀있다는 가정을 할 수 없다. 이런 특수 케이스에 대해서는 1.2.4.에서 자세히 다루어 보도록 하겠다.

마지막으로, stack과 register을 교환한 이후의 코드인 thread_schedule_tail()에 대해 알아 보자. 새로 넘어온 thread에게는 THREAD_RUNNING state를 주며, 이전의 thread가 THREAD_DYING 상태였다면 페이지 메모리를 free하는 작업도 해 준다.

```
thread_schedule_tail (struct thread *prev)
{
    struct thread *cur = running_thread ();

    ASSERT (intr_get_level () == INTR_OFF);

    /* Mark us as running. */
    cur->status = THREAD_RUNNING;

    /* Start new time slice. */
    thread_ticks = 0;

#ifdef USERPROG
    /* Activate the new address space. */
    process_activate ();
#endif

    /* If the thread we switched from is dying, destroy its struct
       thread.  This must happen late so that thread_exit() doesn't
       pull out the rug under itself.  (We don't free
       initial_thread because its memory was not obtained via
       palloc().) */
    if (prev != NULL && prev->status == THREAD_DYING && prev != initial_thread)
    {
        ASSERT (prev != cur);
        palloc_free_page (prev);
    }
}
```

1.2.4. Context switching: new thread인 경우

1.2.3.에서 우리는 `THREAD_RUNNING` state에 한 번이라도 있어 본 두 thread 간의 context switch에 대해 살펴보았다. 또, 같은 방식이 완전히 새로운 thread에 대해서는 적용되기 어려울 것이라는 것도 알아보았다. 이번 단락에서는 위의 문제를 해결하는 방법에 대해 논의할 것이다.

이제 대략적인 아이디어는 다음과 같다. Thread의 시작은 앞서 봤듯 `switch_threads()`의 중반부에서 시작할 것이다. 이 때, 예를 들어 `pop` 명령어에 대응되는 dummy 데이터를 남겨 주고, `ret`를 만날 때는 적당한 함수의 address로 미리 stack을 채워 주면서 새로운 thread의 instruction pointer을 잘 조작할 수 있다는 것이 핵심이다.

1.2.2.에서 살펴 보았던 `thread_create()`함수에서 이해하기 어려웠던 `kf`, `ef`, `sf`이 바로 그 역할을 해 주는 것이었다! 참고가 되도록, 해당 코드를 다시 붙여 넣었다.

```
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    struct thread *t;
    struct kernel_thread_frame *kf;
    struct switch_entry_frame *ef;
    struct switch_threads_frame *sf;
    (...)

    /* Stack frame for kernel_thread(). */
    kf = alloc_frame (t, sizeof *kf);
    kf->eip = NULL;
    kf->function = function;
    kf->aux = aux;

    /* Stack frame for switch_entry(). */
    ef = alloc_frame (t, sizeof *ef);
    ef->eip = (void (*) (void)) kernel_thread;

    /* Stack frame for switch_threads(). */
    sf = alloc_frame (t, sizeof *sf);
    sf->eip = switch_entry;
    sf->ebp = 0;

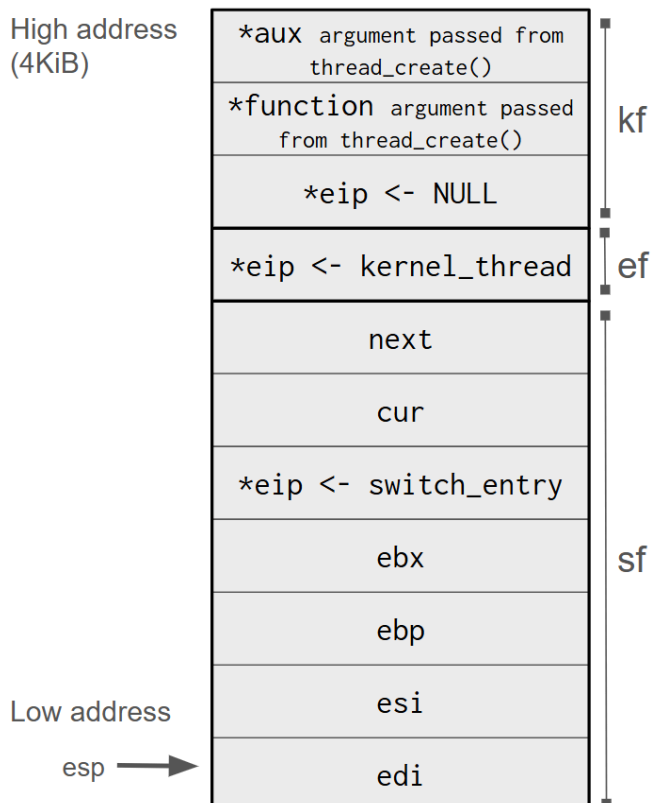
    (...)
}
```

아래에서 `kernel_thread_frame`, `switch_entry_frame`과 `switch_threads_frame`의 struct가 어떻게 정의되어 있는지, 또 `thread_create()` 함수에서 각 frame에 어떤 값을 넣고 있는지를 종합하면 다음과 같은 stack 그림을 그릴 수 있다.

```
struct kernel_thread_frame
{
    void *eip;           /* Return address. */
    thread_func *function; /* Function to call. */
    void *aux;           /* Auxiliary data for function. */
};

struct switch_entry_frame
{
    void (*eip) (void);
};

struct switch_threads_frame
{
    uint32_t edi;        /* 0: Saved %edi. */
    uint32_t esi;        /* 4: Saved %esi. */
    uint32_t ebp;        /* 8: Saved %ebp. */
    uint32_t ebx;        /* 12: Saved %ebx. */
    void (*eip) (void);  /* 16: Return address. */
    struct thread *cur;   /* 20: switch_threads()'s CUR argument. */
    struct thread *next;  /* 24: switch_threads()'s NEXT argument. */
};
```



더 자세히 알아보기 전에, 일련의 과정을 이해하기 위해서는 x86 프로세서의 calling convention에 대해서 알아야 한다. x86의 calling convention은 다음과 같다. 만약 `func(arg1, arg2, arg3);`이라는 함수를 호출해야 한다면, caller function은 `push arg3, push arg2, push arg1` 순서대로 스택에 인자를 저장한 뒤 `call func`을 실행한다. Callee function은 이 convention을 토대로 `%ebp+x` 등의 표현을 이용해 인자에 접근한다.

이제부터, `switch_threads()`의 중반에서부터 시작해서 코드를 따라가며 스택의 변화를 살펴 볼 것이다. 각 stack frame이 어떤 목적으로 만들어졌는지에 집중하며 코드를 읽어 보자.

`switch_threads_frame`은 `context_switch()` 함수에 의해 생기는 부분의 코드들을 모방하여 dummy data 등을 넣은 영역이다. 아래의 `context_switch()` 함수의 `ret`까지 도달하면, 우리의 instruction pointer는 `switch_entry()` 함수로 넘어가고 `%esp`는 `cur`을 가리키게 된다.

```
# Restore stack pointer from new thread's stack.
movl SWITCH_NEXT(%esp), %ecx
movl (%ecx,%edx,1), %esp

# Restore caller's register state.
popl %edi
popl %esi
popl %ebp
popl %ebx
ret
.endfunc
```

아래는 `switch_entry()` 함수이다. 먼저 `addl $8, %esp`를 통해 `context_switch()`의 argument를 버린 뒤, `thread_schedule_tail()` 함수를 실행시킨다. (이 시점에서 thread는 `THREAD_RUNNING` 상태가 될 것이다.) 이후 `ret`를 만나게 되는데, `switch_entry_frame`은 여기서의 `ret`가 `kernel_thread()` 함수를 실행시킬 수 있도록 해주는 역할이다. `kernel_thread()`가 실행되는 시점에서 `%esp`는 `kf`의 가장 낮은 주소를 가리킨다.

```
.globl switch_entry
.func switch_entry
switch_entry:
    # Discard switch_threads() arguments.
    addl $8, %esp

    # Call thread_schedule_tail(prev).
    pushl %eax
.globl thread_schedule_tail
    call thread_schedule_tail
    addl $4, %esp

    # Start thread proper.
    ret
.endfunc
```

`kernel_thread()` 함수는 thread가 목적 함수를 실행시키는 역할을 한다. 또, 목적 함수에서 return이 발생했으면 그 thread를 죽이는 역할도 할 수 있다. 이 때 우리는 `kernel_thread_frame` 덕분에, 현재 실행 시점에서 `kernel_thread()` 함수에 들어가는 인자가 이미 `function`과 `aux`으로 맞아떨어지고 있음을 확인할 수 있다.

```
static void
kernel_thread (thread_func *function, void *aux)
{
    ASSERT (function != NULL);

    intr_enable ();          /* The scheduler runs with interrupts off. */
    function (aux);          /* Execute the thread function. */
    thread_exit ();          /* If function() returns, kill the thread. */
}
```

1.3. Synchronization primitives

이 단락에서는 현재 구현된 핀토스에서 어떻게 synchronization을 수행하고 있는지를 분석한다. 또한 어떤 문제가 존재하고, 이를 해결하기 위해 어떤 방법을 사용할 수 있을지 간략히 살펴본다.

Synchronization과 관련된 함수들은 threads/synch.h, threads/synch.c 에 정의되어 있고, semaphore, lock, 그리고 monitor에 대해 살펴본다.

1.3.1. Semaphore

```
/* A counting semaphore. */
struct semaphore
{
    unsigned value;          /* Current value. */
    struct list waiters;     /* List of waiting threads. */
};
```

코드에서 semaphore는 구조체 형식으로 선언되었으며, 현재 semaphore값과 실행되려다가 semaphore의 값이 0이라 실행하지 못하고 기다리고 있는 스레드들의 리스트를 갖고 있다.

- sema_down()

```
void
sema_down (struct semaphore *sema)
{
    enum intr_level old_level;

    ASSERT (sema != NULL);
    ASSERT (!intr_context ());

    old_level = intr_disable ();
    while (sema->value == 0)
    {
        list_push_back (&sema->waiters, &thread_current ()->elem);
        thread_block ();
    }
    sema->value--;
    intr_set_level (old_level);
}
```

Semaphore의 down(P) 연산을 수행하는 함수이다. 만약 sema의 value가 0이라면 이 스레드가 실행대선 안되므로 while문으로 막아두고, 해당 semaphore의 대기자 명단에 현재 스레드를 추가한다. 이때 해당

스레드는 block 상태로 바뀌고, sema의 값이 0이 아닌 값으로 바뀌어 실행 가능해지면 value를 1 감소시킨다. 이 과정들이 수행될 때 atomicity를 보장하기 위해 `intr_disable()`과 `intr_set_level()`로 감싸놓았다.

- `sema_up()`

```
void
sema_up (struct semaphore *sema)
{
    enum intr_level old_level;

    ASSERT (sema != NULL);

    old_level = intr_disable ();
    if (!list_empty (&sema->waiters))
        thread_unblock (list_entry (list_pop_front (&sema->waiters),
                                                struct thread, elem));
    sema->value++;
    intr_set_level (old_level);
}
```

Semaphore의 up(V) 연산을 구현한 함수이다. 접근하려는 semaphore의 대기자 리스트가 비어있지 않다면 가장 앞의 스레드를 pop하고(FIFO 방식) 해당 스레드를 unblock 한다. 이후 semaphore의 값을 1 증가시킨다. 마찬가지로 이 명령을 실행하는 동안에는 interrupt를 disable 시키고, 명령이 다 수행되면 다시 활성화시킨다.

1.3.2. Lock

```
/* Lock. */
struct lock
{
    struct thread *holder; /* Thread holding lock (for debugging). */
    struct semaphore semaphore; /* Binary semaphore controlling access. */
};
```

Lock은 semaphore와 비슷하지만, 초기값이 1로 고정되어 있고, lock을 down 시킨 스레드만이 해당 lock을 up 시킬 수 있다는 차이점이 존재한다. 따라서 lock의 구조를 보면 이 lock을 acquire 하고 있는 thread와 binary semaphore로 사용될 semaphore가 선언되어 있다.

- lock_init()

```
void
lock_init (struct lock *lock)
{
    ASSERT (lock != NULL);

    lock->holder = NULL;
    sema_init (&lock->semaphore, 1);
}
```

Lock을 초기화 할 때는 holder를 null로 하고, semaphore를 1로 초기화 한다.

- lock_acquire()

```
void
lock_acquire (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (!lock_held_by_current_thread (lock));

    sema_down (&lock->semaphore);
    lock->holder = thread_current ();
}
```

Lock을 acquire하려 할 때 사용되는 함수로, semaphore의 값을 1 감소시키기 전에 3가지를 검사한다. 먼저 lock이 유효한 lock인지를 검사하고, intr_context()가 false를 반환하는지 검사한다. 이는 external interrupt를 processing하는 중에 lock을 얻게 되면 데드락 현상이 발생할 수 있기 때문이다. 마지막으로 자기 자신이 acquire 하고 있는 lock을 acquire 하지 못하게 막는 것으로, 핀토스에서 사용되는 lock이 재귀적이지 않기 때문에 필요한 검사이다. 3개의 검사를 통과했다면 semaphore의 값을 1감소시키고, holder를 현재 스레드로 저장한다.

- lock_release()

```
void
lock_release (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (lock_held_by_current_thread (lock));

    lock->holder = NULL;
    sema_up (&lock->semaphore);
}
```

Lock을 release하는 함수로, acquire할 때와는 반대로 현재 스레드가 해당 lock을 점유하고 있어야만 release 할 수 있도록 검사를 한다. 이는 위에서 설명했듯 acquire한 스레드만이 해당 lock을 놓아줄 수 있기 때문이다. 검사를 통과하면 holder를 없애고 semaphore의 값을 1증가 시킨다.

1.3.3. Monitor

Lock이 semaphore를 포함했다면 monitor는 lock을 포함하는 synchronization primitive이다.

Pintos에서 monitor는 lock과 condition variables로 이루어진다.

```
/* Condition variable. */
struct condition
{
    struct list waiters;      /* List of waiting threads. */
};
```

Condition variable에는 대기 중인 스레드들이 저장된다.

- cond_init()

```
void
cond_init (struct condition *cond)
{
    ASSERT (cond != NULL);

    list_init (&cond->waiters);
}
```

다음과 같이 초기화를 시키며, waiters를 빈 리스트로 초기화 한다.

- cond_wait()

```
void
cond_wait (struct condition *cond, struct lock *lock)
{
    struct semaphore_elem waiter;

    ASSERT (cond != NULL);
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (lock_held_by_current_thread (lock));

    sema_init (&waiter.semaphore, 0);
    list_push_back (&cond->waiters, &waiter.elem);
    lock_release (lock);
    sema_down (&waiter.semaphore);
    lock_acquire (lock);
}
```

이 함수는 'sema_init' 을 통해 semaphore를 0으로 초기화한다. 이후 자신의 semaphore_elem을 condition의 waiters 목록에 추가한다. 데드락을 방지하기 위해 lock을 풀고, sema_down을 호출해 스레드를 block한다. 이 스레드는 이후 cond_signal() 함수가 호출되면 block에서 풀려나고, 다시 lock을 acquire한다.

- cond_signal()

```
void
cond_signal (struct condition *cond, struct lock *lock UNUSED)
{
    ASSERT (cond != NULL);
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (lock_held_by_current_thread (lock));

    if (!list_empty (&cond->waiters))
        sema_up (&list_entry (list_pop_front (&cond->waiters),
                                             struct semaphore_elem, elem)->semaphore);
}
```

이 함수는 condition의 대기자 목록에서 가장 오래된(FIFO) 스레드를 pop하고 sema_up을 통해 block을 해제한다.

- cond_broadcast()

```
void
cond_broadcast (struct condition *cond, struct lock *lock)
{
    ASSERT (cond != NULL);
    ASSERT (lock != NULL);

    while (!list_empty (&cond->waiters))
        cond_signal (cond, lock);
}
```

Condition의 대기 중인 스레드들을 순차적으로 wake 시키는 함수이다.

2. Design Proposal

2.1 Alarm clock

이 단락에서는 alarm clock을 어떻게 구현할지에 대한 계획을 설명한다. `timer_sleep()`은 스레드를 주어진 시간 동안 스케줄링 되지 않도록 하는 것을 목표로 한다. 현재 구현된 핀토스에서는 이를 busy waiting 방식으로 처리하는데, 매 `timer_interrupt`마다 몇 tick이 지났는지 확인하고 시간이 지나지 않았다면 yield 하는 메커니즘이다. 시간이 지나지 않아 어차피 양보를 해야하는 스레드임에도 불구하고 ready queue에 존재해서 cpu에게 자신을 계속 확인하게 한다는 문제점이 존재한다. Cpu가 정해진 시간 동안은 이 스레드를 아예 스케줄링에 고려하지 않을 수 있도록 스레드를 block 해야 한다. 자세한 구현은 아래와 같이 계획했다.

2.1.1 Data structure

스레드가 언제 깨어날지에 대한 정보를 스레드에 포함시켜야 한다.

```
/* threads/thread.h */
struct thread
{
    ...
    int64_t tick_to_wake; // Add this line
    ...
}
```

또한 동시에 여러 스레드가 sleep 상태일 수 있으므로 이 스레드들을 관리할 리스트가 timer.c 에 필요하다.

```
/* devices/timer.c */
...
static struct list sleeping_list; // Added
...
```

2.1.2 Algorithm

```
/* devices/timer.c */
void
timer_sleep (int64_t ticks)
{
    Disable interrupt
    1. Determine the time(tick) that this thread should wake up -> save at
    tick_to_wake
```

```

2. Append current thread to the sleeping_list
3. Call thread_block()
   Enable interrupt
}

```

위와 같은 의사코드를 수행하도록 timer_sleep()을 변경한다. 언제 깨워야 하는지를 저장하고, sleeping_list에 자신을 추가한 후 thread_block()을 호출해 자신을 block한다. 시간이 되어 wake up을 시키는 부분은 timer_interrupt()에서 수행한다. sleeping_list를 변화시키는 중간에 interrupt가 일어나면 race condition이 발생할 수 있으므로 interrupt를 disable한 상태에서 실행 되도록 해야 한다.

```

/* devices/timer.c */
/* Timer interrupt handler. */
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    thread_tick ();
    1. Walk through all the elements in sleeping_list and compare its tick_to_wake
    and ticks
    2. If there are threads need to be waken up, call
    thread_unblock(thread_that_should_wake_up) and remove it(pop) from the
    sleeping_list
}

```

기존 코드에 추가적으로 위와 같은 명령을 수행하는 코드를 작성하면 된다. 깨울 스레드가 있는지 순회하며 확인하고, 있다면 thread_unblock()을 실행해 thread의 state을 blocked에서 ready로 바꾼다. 이 함수는 매 tick마다 실행되므로 state을 변경할 스레드가 있는지 검사하기에 적절하다.

실제 구현을 할 때는 여기에 덧붙여 race condition이 일어날 가능성도 해결해줘야 한다. 서로 다른 두 함수가 sleeping_list 라는 static 변수를 바꿀 수 있으므로 여기서 race condition이 일어날 수 있다. 따라서 sleeping_list를 변화시킬 때는 전후에 interrupt disable/enable을 시켜 race condition을 방지하도록 해야 한다.

2.2. Priority Scheduling

먼저, 구현해야 하는 내용을 추상적으로 분류하면 다음과 같다.

- 고려할 priority가 추가되는 경우 (새로운 thread의 생성, semaphore이나 lock을 기다리는 thread 추가 등)
 - Thread의 ready list, semaphore의 wait list에 추가할 때 정렬해서 넣으면 된다.
 - Thread의 ready list의 맨 앞에 오는 경우, 현재 실행되는 thread와도 비교해서 더 높은 priority를 가지는 thread를 실행 thread로 바꿔야 한다.
- 특정 priority가 수정되는 경우 (현재 돌아가는 thread가 스스로 priority를 수정한 경우, priority donation을 받은 경우)
 - 현재 thread의 priority를 수정할 수 있는 로직이 필요하다.
 - Semaphore 등의 Wait list가 정렬을 유지하도록 수정해야 한다. thread의 ready list도 비슷한데, priority가 바뀐 thread가 현재 실행중이었을 경우, 더이상 최고의 priority가 아니면 yield까지 해주어야 한다.

또한, priority inversion을 방지하기 위한 priority donation을 잘 설계해야 한다. H, M, L은 thread라고 하고, priority는 $H > M > L$ 이라 하자.

- 발생할 수 있는 문제: L이 lock을 가지고 있다고 하자. 이후 H가 등장해서, 이후 L이 소유한 lock을 받기 위해 blocked 상태로 전이되었다고 하자. 그 후 M이 등장한다면, M이 완료될 때까지 H는 실행 상태가 될 수 없다. 하지만 H의 priority가 더 높기 때문에, 이는 분명한 문제이다.
- 기본 donation: H가 사용해야 하는 lock이 L에 소유되어 있을 경우, H는 L에게 자신의 priority를 빌려준 후 block 상태로 넘어 간다. (양도받은 priority를 effective priority라고 하자.) L이 lock을 반납할 때, 후원받은 priority도 같이 반납해야 한다.
- Multiple donation: H과 M이 모두 L이 소유한 lock을 사용하고 싶을 경우, H과 M 중 높은 priority를 양도받아 사용한다. 중요한 점은 H과 M 또한 donation 등으로 중간에 priority가 바뀔 수 있어서, 저장뿐만 아니라 업데이트도 가능해야 한다. 이를 해결하는 좋은 방법은, lock의 wait list가 업데이트될 때마다 lock의 소유자인 L의 effective priority를 업데이트해주는 방식이다.
- Nested donation: L이 lock l을 소유하고, M은 lock m을 소유하고 있다고 하자. M이 l을 사용하고 싶고 L이 m을 사용하고 싶은 경우, H, M, L의 (priority, effective priority)는 각각 (H, H), (M, H), (L, H)이 되며 L이 실행되고 있어야 한다. 이를 구현하는 좋은 방법은 thread가 기다리고 있는 lock들의 link를 계속 따라가며 effective priority를 업데이트해 주는 것이다. 예를 들어, M이 L에게 priority를 빌려 줄 때 M은 l에 대한 정보를 저장한다. 이후, H가 실행되어 m을 접근하고 싶어할 때, L은 M에게 priority를 빌려 주고, M은 l의 주인인 L의 priority도 같이 업데이트 시켜 주는 것이다. 만약 이 nesting이 더 길어지더라도 비슷한 로직으로 구현 가능하다.

위의 내용을 바탕으로, 추가하거나 수정해야 할 data structure을 알아 보자.

- `struct thread`에 `int effective_priority`를 추가해야 할 것이다. 이 값은 lock에 구현되는 priority donation을 위해 도입되는 것으로, thread가 가진 모든 lock의 wait list effective priority와 자신의 그냥 priority 중 가장 큰 값을 취하면 될 것이다.
- `struct thread`에 `struct list locks_holding`이라는 리스트를 추가해야 할 텐데, 이는 특정 thread가 지금 가지고 있는 lock들을 모은 list이다. 위에서의 `effective_priority`의 산출 방식을 생각해 보았을 때 필요한 data structure이다.

또, 추가하거나 수정할 function을 생각해 보자. 우선, priority간의 값을 비교해야 할 때에는 `effective_priority`를 통해 계산되어야 할 것이다.

- `thread_unblock()`, `thread_yield()`, `sema_down()`: ready list 또는 wait list에 element를 추가하는 함수이다. 이 함수들이 element를 추가할 때 정렬해 넣는 알고리즘을 추가 구현하는 것이 중요하다. 이 때, `thread_unblock()`으로 다시 ready가 되는 thread의 priority가 제일 클 수도 있는 경우를 대비해, 이 함수에서도 `thread_yield()`을 부르는 것이 좋겠다.
- Priority를 수정하는 함수 `thread_update_priority()`를 추가해줄 수 있다. `thread_update_priority()`는 thread와 새로운 priority 값을 argument로 받아, priority를 수정한 뒤 그 thread에 속한 ready list나 wait list 등의 순서를 재정렬하고, ready list의 가장 큰 값과 비교해서 effective priority가 작아지면 다시 `thread_yield()`을 불러 우선순위가 제일 높은 thread로 바꾸는 작업을 해 줄 수 있다.
- Thread의 effective priority를 수정하는 함수 `thread_update_effective_priority()`를 추가해줄 수 있다. `thread_update_effective_priority()`는 thread 포인터 하나를 input으로 받아, 그 thread의 `locks_holding`값에 있는 각각의 lock의 wait list에 대해 effective priority를 조사해, 그 최댓값과 자신의 priority 중 최댓값을 `effective_priority`로 저장하는 로직으로 짤 수 있겠다. `thread_update_priority()`처럼 list들을 재정렬할 때도 해야 할 것으로 보인다. 이 함수는 lock이 수정되는 `lock_acquire()` 함수에서 호출되어야 하며, `effective_priority`가 실제로 업데이트될 때마다 재귀적으로 불러줘야 할 것을 예상할 수 있다.
- Priority donation은 특정한 함수로 작성하진 않고, effective priority의 업데이트를 통해 간접적으로 이루어질 것으로 생각된다.

2.3 Advanced Scheduler

보다 유동적으로 스레드의 우선순위(priority)를 결정지을 수 있도록 하려면 mlfqs 방식, 즉 Multi-Level Feedback Queue Scheduling 방식을 구현해야 한다. 이는 pintos에 구현이 되어 있지 않으며 스켈레톤 코드만 일부 존재한다.

먼저 mlfqs 방식 중 하나인 4.4BSD의 주요 특징들을 살펴보면 다음과 같다.

1. 실행가능하지만 대기중인 스레드들을 담은 큐가 여러개 있다.
2. 우선순위가 더 높은 큐와 낮은 큐가 존재하며 비어 있지 않은 큐들 중에서 가장 우선순위가 높은 큐의 스레드를 스케줄러가 선택한다.
3. 가장 우선순위가 높은 큐에 여러개의 스레드가 존재한다면 round robin 방식으로 스레드들을 실행시킨다.
4. 우선순위는 스레드가 만들어질 때, 그리고 매 4-tick마다 모든 스레드에 대해 다시 계산된다.

다음으로 스레드의 우선순위가 구체적으로 어떻게 계산되는지 살펴본다. 계산에 사용되는 요인들은 다음과 같다.

1. nice
2. load_avg
3. recent_cpu

nice

이 변수는 -20에서 20 사이의 정수를 가질 수 있으며, 얼마나 nice한지, 즉 우선순위를 얼마나 다른 스레드에 양보하는지에 대한 정도를 나타낸다. Nice가 20이면 자신의 우선순위를 더 낮추고 -20이면 자신의 우선순위를 높이려는 것으로 해석할 수 있다.

```
/* Sets the current thread's nice value to NICE. */
void
thread_set_nice (int nice UNUSED)
{
    /* Not yet implemented. */
}
```

위의 함수를 구현해야 하며, 매개변수로 받은 값으로 스레드의 nice 값을 변경하도록 해야 한다.

```
/* Returns the current thread's nice value. */
int
thread_get_nice (void)
{
    /* Not yet implemented. */
    return 0;
}
```

```
}
```

위의 함수는 스레드의 nice 값을 반환하기만 하면 되는 간단한 함수이다.

load_avg

이 값은 지난 1분간 몇 개의 스레드들이 ready to run 상태였는지를 나타내는 값이다. 이는 스레드마다 결정되는 값이 아니라 시스템 전체적으로 계산되는 값이다.

$$\text{load_avg} = (59/60) * \text{load_avg} + (1/60) * \text{ready_threads}$$

위의 식으로 계산되며, 계산은 매초 이루어진다. 초기값은 0이다. 구현적으로 $\text{timer_ticks()} \% \text{TIMER_FREQ} == 0$ 인 경우에 계산되어야 한다.

```
/* Returns 100 times the system load average. */
int
thread_get_load_avg (void)
{
    /* Not yet implemented. */
    return 0;
}
```

위의 함수를 통해 구현해야 하며, 100을 곱한 후 반환해야 한다. 이는 load_avg가 실수 값을 가지는데 반해 반환 형식은 int이기 때문에, 소수점 아래 두자리까지 값을 보전하기 위함이다.

recent_cpu

최근에 더 많은 cpu 사용시간을 보인 스레드는 cpu가 해야할 일이 매우 많은 스레드로 생각할 수 있고, 따라서 더 높은 우선순위를 가져야 한다. Recent_cpu는 이 점을 반영하는 변수로, exponentially weighted moving average 방식을 이용해 구한다. 이를 이용한 식은 다음과 같다.

$$\text{recent_cpu} = (2 * \text{load_avg}) / (2 * \text{load_avg} + 1) * \text{recent_cpu} + \text{nice}$$

초기값은 0으로 설정되며 load_avg와 nice를 이용해서 계산하는 것을 볼 수 있다. 또한 recent_cpu는 매 timer interrupts마다 1증가하며, 위의 식은 매초 계산된다. Load_avg와 마찬가지로 매초 계산되는 것을 구현하기 위해 $\text{timer_ticks()} \% \text{TIMER_FREQ} == 0$ 인 경우에만 계산되도록 해야 한다.

```
/* Returns 100 times the current thread's recent_cpu value. */
int
thread_get_recent_cpu (void)
{
    /* Not yet implemented. */
    return 0;
}
```

위의 함수를 이용해 구현해야 하며, `thread_get_load_avg()` 와 마찬가지로 100을 곱한 값을 반환하도록 해 `int`로 반환할 때도 손실이 최소한으로 일어나게 한다.

이러한 연산은 필연적으로 부동소수점 연산을 필요로 하는데, 핀토스는 커널의 단순함과 빠른 실행 속도를 위해 부동소수점 연산을 지원하지 않는다. 따라서 부동소수점 연산을 근사하는 정수 기반의 알고리즘이 필요하다. 이를 위해 `fixed_point.h` 라는 별도의 파일을 만들 계획이다. 이 파일 내부적으로는 17.14 방식의 방법을 사용할 것이다.

32비트 정수의 rightmost 14-bits를 fractional-bits로 사용하고 MSB를 sign-bit으로 사용한다. 가운데의 17-bits는 정수부분을 나타내도록 사용할 수 있다. 구현시 `pintos manual`의 Appendix B.6의 마지막 부분을 참고할 것이다.

Convert <code>n</code> to fixed point:	<code>n * f</code>
Convert <code>x</code> to integer (rounding toward zero):	<code>x / f</code>
Convert <code>x</code> to integer (rounding to nearest):	<code>(x + f / 2) / f</code> if <code>x >= 0</code> , <code>(x - f / 2) / f</code> if <code>x <= 0</code> .
Add <code>x</code> and <code>y</code> :	<code>x + y</code>
Subtract <code>y</code> from <code>x</code> :	<code>x - y</code>
Add <code>x</code> and <code>n</code> :	<code>x + n * f</code>
Subtract <code>n</code> from <code>x</code> :	<code>x - n * f</code>
Multiply <code>x</code> by <code>y</code> :	<code>((int64_t) x) * y / f</code>
Multiply <code>x</code> by <code>n</code> :	<code>x * n</code>
Divide <code>x</code> by <code>y</code> :	<code>((int64_t) x) * f / y</code>
Divide <code>x</code> by <code>n</code> :	<code>x / n</code>